```
  def save_widget!(params) do
    widget = WidgetStore.create_widget!(params)
    WidgetSync.enqueue!(widget)
  end
end
```

The alias keyword refers to the provided module name using the last part of its name. So in this example, you can write WidgetStore instead of the fully qualified name.

You can specify multiple modules at once by using the syntax alias ParentModule.{First, Second}. This syntax has mixed reviews in the community—many people prefer to write each alias on its own line.

Import

The import keyword scopes a module's public functions into the current module. The function effectively becomes local to your module at that point. A common example of this is the Ecto.Changeset module:

```
defmodule ExampleModule do
  import Ecto.Changeset

  def changeset(attrs) do
    %MySchema{}
    |> cast(attrs, [:field])
    |> validate_required([:field])
  end
end
```

In this example, cast/3 and validate_required/2 are functions defined in the Ecto.Changeset module. (The arity may be higher than you are expecting. Remember that the pipe operator provides the first argument.)

To keep our code clean, we import the Ecto.Changeset module's functions and then use them directly. This syntax is most useful for a domain-specific language (DSL). It's common in Ecto primarily because Ecto is a DSL over database operations. We'll be using Ecto in Chapter 6, Persisting Data with Ecto, on page 87.

Require

If you want to use a macro function in your module, then you must require the module before you do so. If you forget to require the macro's module first, Elixir will give you a helpful warning.